High-Level Scheduling of Energy Optimal Trajectories

Oskar Wigström, Bengt Lennartson, *Member, IEEE*, Alberto Vergnano, and Claes Breitholtz

***Abstract*—The reduction of energy consumption is today addressed with great effort in manufacturing industry. In this paper, we improve upon a previously presented method for robotic system scheduling. By applying dynamic programming to existing trajectories, we generate new energy optimal trajectories that follow the same path but in a different execution time frame. With this new method, it is possible to solve the optimization problem for a range of execution times for the individual operations, based on one simulation only. The minimum energy trajectories can then be used to derive a globally energy optimal schedule. A case study of a cell comprised of four six-link manipulators is presented, in which energy optimal dynamic time scaling is compared to linear time scaling. The results show that a significant decrease in energy consumption can be achieved for any given cycle time.**

***Note to Practitioners*—In robotic manufacturing systems, much energy is wasted due to an adopted minimum time policy for robot operations. This paper presents a method for producing energy efficient operations as well as an optimization model for scheduling these operations in a robot cell. Implementation requires a flexible robot controller which allows manipulation of speed and acceleration profiles. The results are also of interest to industrial robot manufacturers, who have full control over the robot controller and would like to offer energy efficient solutions to their customers.**

***Index Terms*—Optimization methods, scheduling, trajectory.**

I. Introduction

ENERGY efficiency is a very important design driver for robots and other moving devices in manufacturing systems. In fact, oil and electricity prices are rapidly increasing, and also companies will have to undergo new environmental policies for a sustainable future. This can be accomplished by research on new efficient equipment with, for instance, regenerative braking, energy storages, and lightweight solutions. As a short time strategy, however, this paper focuses on existing hardware solutions and presents a method for the optimization of their usage.

The final performances of a robotic manufacturing system result from the synergistic interaction between mechanics, control, and software [1]. Thus, the optimization of such systems should exploit different research fields. Energy optimization of mechatronic devices is well investigated in [2]–[5]. Energy optimal trajectories for robot applications is a big research field itself, see, e.g., [6], [7], and [8]. From a system design perspective, a selection and matching of efficient design solutions for pre-defined operations is studied in [9]–[11]. The optimization of the scheduling supervisor of the overall system is a promising area, although most works are concerned with cycle time, as in [12] and resented in [14], where idle time between the operations is used to reduce velocities and accelerations, without concern to the energy consumption.

A method for decreasing the energy consumption in production systems was presented in [15]. It exploits idle time caused by mutual exclusion caused by for example two robots work in a shared zone, or simply preconditions on operations. One of the building blocks in the optimization method is the calculation of the energy consumption $E_\ell(t_{f\ell})$ for each individual robot operation ℓ as a pseudo-polynomial function of its execution time $t_{f\ell}$. These local energy functions are then used to determine the globally optimal execution times $t^*_{f\ell}$ for all operations, such that the overall energy consumption of the system is minimized. The energy functions in [15] are based on a uniform time scaling of the robot trajectory, i.e., a linear stretching of the position profile. In this paper, we improve the method in [15] introducing a dynamic time scaling, in order to deliver an energy optimal trajectory.

In [16], we presented preliminary results for improved method. Using dynamic programming (DP), we solve the trajectory planning problem for a range of execution times simultaneously. An example consisting of a two-joint planar robot arm following a simple trajectory was used to compare dynamic and linear scaling. Results showed a more than 5%, dynamic scaling would use at least 4% less energy than linear scaling. In this paper, DP for generation of optimal energy functions is combined with high-level scheduling. The scheduling problem is, as in [15], modeled with mixed integer linear constraints, with a nonlinear cost function expressing the energy consumption of the system. A case study with four industrial robots operating in the same cell is also reported.

This paper is structured as follows. Section II contains an introduction to trajectory planning as well as the optimization

Manuscript received September 02, 2011; revised March 23, 2012; accepted May 02, 2012. Date of publication May 23, 2012; date of current version December 18, 2012. This paper was recommended for publication by Associate Editor M. Nakano and Editor K. Goldberg upon evaluation of the reviewers' comments. This work was carried out at the Wingquist Laboratory VINN Excellence Center within the Area of Advance-Production at Chalmers University of Technology, supported in part by the Swedish Governmental Agency for Innovation Systems (VINNOVA) and the Swedish Research Council.

O. Wigström and B. Lennartson are with the Automation Research Group, Department of Signals and Systems, Chalmers University of Technology Göteborg, 412 96, Sweden (e-mail: oskar.wigstrom@chalmers.se; bengt.lennartson@chalmers.se).

A. Vergnano is with the Department of Mechanical and Civil Engineering, University of Modena and Reggio Emilia, Modena 41125, Italy (e-mail: alberto.vergnano@unimore.it).

C. Breitholtz is with the Automatic Control Research Group, Department of Signals and Systems, Chalmers University of Technology, Göteborg, 412 96, Sweden (e-mail: claesbr@chalmers.se).

Digital Object Identifier 10.1109/TASE.2012.2198816

1545-5955/$31.00 © 2012 IEEE
Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on October 19,2026 at 19:03:37 UTC from IEEE Xplore. Restrictions apply.

model and the algorithm used to solve it. Section III briefly covers the scheduling problem. Section IV presents the results from a four robot test case and, finally, in Section V conclusions are drawn along with a brief discussion.

II. Trajectory Planning

A trajectory planning problem can be described as generating the set of control inputs that will move a manipulator along a predefined geometric path without violating any dynamic or kinematic constraints. Usually, trajectory planning problems are concerned with the optimization of some cost function, most often comprised of time, torque, jerk, or a weighted combination of these.

There are a large number of approaches to the minimum energy problem. Solutions include DP [17], and later iterative DP [18], parameterized b-splines [19], Pontryagin's maximum principle [20], among others, were used. Some of the later methods allow for constraints on the jerk and result in a continuous acceleration. In some cases, a simplified cost function is proposed, for example only minimizing the squared joint acceleration. An excellent overview of the last three decades can be found in [21].

In contrast to [17], which employs a weighting between cost and time, our method includes elapsed time in the optimization model. This implies that, while [17] is based on free final time in the optimization, our model can generate solutions for specific final times. Note that, DP, as opposed to other methods, puts no bounds on the plant model complexity or objective function. The main advantage of DP in this context, however, is that it yields optimal solutions for the entire grid. In our method, the optimization model is formulated in such a way that all execution (final) times are included in the grid, and thus the DP optimization only needs to be run once. Also, note that since the optimization is based on a single scaling parameter, the dimensionality will be unchanged for additional robot joints as well as more intricate energy models.

A. Problem Formulation

As previously established, for each operation, a trajectory planning problem needs to be solved for a range of execution times. Since this method is applied to one operation at a time, the operation index i is now discarded. Solving a trajectory planning problem entails finding the input torques required to move a manipulator along a predefined geometric path, while upholding its dynamical constraints. The joint torques of the manipulator can be expressed by a Lagrange formulation, see [22, pp. 131–140]. Einstein summation convention is used, as in [17], where an index appearing in both the subscript and the superscript of a term constitutes a summation over its elements. The torque, T_i acting on the ith joint can be expressed as

$$T_i = J_{ij}\ddot{q}^j + C_{ijk}\dot{q}^j\dot{q}^k + F_{ij}\dot{q}^j + G_i \tag{1}$$

where J is the inertia matrix, C the tensor of centrifugal and Coriolis coefficients, F the viscous friction matrix, G the gravitational vector, and q^i the angular position of joint i. Note that J, C, and G are all functions of q. Also, $\dot{q}^i$ and $\ddot{q}^i$ represent the first and second time derivative of q^i.

Let the geometric path be defined by a function $q_0(\tau)$, a parameterized curve dependent on one single variable $\tau(t)$. In this paper, the time optimal trajectory is used to define q_0 and its derivatives. This implies that τ is the time scale for the time optimal trajectory, q_0. For example, defining $\tau = t$ would result in the time optimal trajectory. Modeling tools such as ABB RobotStudio [23] can be used to generate q_0. The relationship between q and q_0 can be expressed as

$$q(t) = q_0(\tau(t)), \qquad 0 \le \tau \le \tau_f \tag{2}$$

where $\tau(t)$ is a monotonically increasing function with a starting value of 0 and final value τ_f, where τ_f in our case corresponds to the time optimal execution time. If $\tau(t_f) = \tau_f$, then t_f is the new final execution time of the dynamically scaled operation. The derivatives of q_0 with respect to τ are the same as those of the time optimal trajectory with respect to time. Differentiating (2) with regard to time yields expressions for speed and acceleration which are needed for computing the cost function and upholding constraints

$$\dot{q}^i(t) = \frac{dq_0^i(\tau)}{d\tau}\dot{\tau} \tag{3}$$

$$\ddot{q}^i(t) = \frac{dq_0^i(\tau)}{d\tau}\ddot{\tau} + \frac{d^2q_0^i(\tau)}{d\tau^2}\dot{\tau}^2. \tag{4}$$

Further, combining (3) and (4) with (1) results in an expression for the torque as a function of τ and q_0

$$\begin{aligned} T_i = {} & J_{ij}\frac{dq_0^j(\tau)}{d\tau}\ddot{\tau} + J_{ij}\frac{d^2q_0^j(\tau)}{d\tau^2}\dot{\tau}^2 + \\ & + C_{ijk}\frac{dq_0^j(\tau)}{d\tau}\frac{dq_0^k(\tau)}{d\tau}\dot{\tau}^2 + \\ & + F_{ij}\frac{dq_0^j(\tau)}{d\tau}\dot{\tau} + G_i. \end{aligned} \tag{5}$$

The optimization procedure is also subject to a number of dynamic constraints, such as limits on torques, accelerations and speeds. This can be implemented using a barrier function, or since the original trajectory is assumed to be time optimal, by adding the constraint $\dot{\tau} \le 1$.

Considering the operation execution times relevant to this paper, as in [7] and [24], the energy consumption of an AC permanently excited synchronous motor can be expressed by the following simplified voltage and current models:

$$\begin{aligned} V_i(t) &= R_iI_i(t) + K_{V,i}K_{R,i}\dot{q}_i(t) \\ I_i(t) &= \frac{T_i(t)}{(K_{T,i}K_{R,i})} \end{aligned} \tag{6}$$

where $I_i(t)$ and $V_i(t)$ are the equivalent DC current and voltage of the ith rotor, R_i is the stator resistance, $K_{V,i}$ is the electrical (back emf) constant, $K_{R,i}$ is the transmission gear ratio, and $K_{T,i}$ is the equivalent torque constant. With (6) defined, we can express the power of each motor as

$$P_i(t) = V_i(t)I_i(t) = \frac{R_i}{K_{T,i}^2K_{R,i}^2}T_i^2(t) + \frac{K_{V,i}}{K_{T,i}}T_i(t)\dot{q}_i(t). \tag{7}$$

An arbitrary cost function can be used, but in this paper it is of interest to examine minimum energy trajectories for specific

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on October 19,2026 at 19:03:37 UTC from IEEE Xplore. Restrictions apply.

execution times t_f. Integrating (7) gives an expression for the total energy consumption, the cost that is to be minimized

$$E(t_f) = \int_0^{t_f} \left(\sum_{i=1}^{n} P_i(t) \right) dt. \tag{8}$$

Here, n is the number of joints. With the power and torque defined as in (7) and (5), the cost E is a functional of q_0 and τ. Since the former is known, solving the optimization problem is a matter of finding τ, while minimizing the cost and upholding dynamic constraints. The optimal cost $E^*(t_f)$ is the local energy function sought for each operation.

B. Optimization Model

As mentioned, solving the trajectory planning problem entails finding the τ in (2) that minimizes a given cost function, such as (8). Define the second derivative of τ as

$$\ddot{\tau}(t) = u(t) \tag{9}$$

where $u(t)$ is a control input. Also, introduce a time-varying sampling time h_k that affects the time updates as

$$t_{k+1} = t_k + h_k \tag{10}$$

and let the input variable be piecewise constant during the sampling intervals, i.e.,

$$u(t) = u(t_k), \qquad t_k \leq t < t_{k+1}. \tag{11}$$

The decision to use a piecewise constant $\ddot{\tau}$ is an abstraction that will restrict the dimensionality of the problem to two. Even though it will introduce small discontinuities in the acceleration through (4), these minor artifacts can be considered marginal. Discretization of (9), with a sampling period h_k and constant control input as in (10) and (11), gives the discrete state space model

$$\begin{bmatrix} \tau_{k+1} \\ \nu_{k+1} \end{bmatrix} = \begin{bmatrix} 1 & h_k \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \tau_k \\ \nu_k \end{bmatrix} + \begin{bmatrix} \frac{h_k^2}{2} \\ h_k \end{bmatrix} u(t_k) \tag{12}$$

where $\nu = \dot{\tau}$ and, for simplicity, we introduce $\tau_{k+1} = \tau(t_{k+1})$, $\tau_k = \tau(t_k)$ and $\nu_{k+1} = \nu(t_{k+1}), \nu_k = \nu(t_k)$. The minimization of (8), including this discrete time model of the time function $\tau(t)$ can be solved with DP, but for computational reasons discussed later, it is convenient to reformulate the problem. Since τ is monotonically increasing it is possible, instead of taking steps along the t axis in each iteration, to take steps along the τ axis and let (10) act as a discrete state equation. Define

$$h_k \nu_k + h_k^2 \frac{u(t_k)}{2} = \Delta_k \tag{13}$$

where Δ can be regarded as a user defined sampling period or gridding of τ. If (13) is inserted into (12), then in every step k, τ will be updated as

$$\tau_{k+1} = \tau_k + \Delta_k. \tag{14}$$

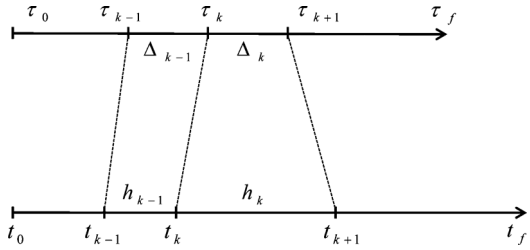


Fig. 1. Uniformly spaced τ mapped onto t.

Equation (13) can also be manipulated into an expression for the control signal, $u(t_k) = 2(\Delta_k - h_k \nu_k)/h_k^2$. Inserting this expression for $u(t_k)$ into the bottom equation of (12) gives a new state equation for ν. Regarding the sampling time, h_k as the new control signal and letting (10) act as a state-space equation leads us to the reformulated discrete state-space model

$$\begin{bmatrix} t_{k+1} \\ \nu_{k+1} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} t_k \\ \nu_k \end{bmatrix} + \begin{bmatrix} h_k \\ \frac{2\Delta_k}{h_k} \end{bmatrix}. \tag{15}$$

From here, DP can be applied to solve the discrete optimal control problem. The relation between (14) and (10) can be seen as a mapping of τ onto t, which is illustrated in Fig. 1.

C. Dynamic Programming

DP is an optimization method which can be applied to problems where a series of decisions need to be made and the dynamics of the system can be determined from any location within a state space. For a detailed account of the theory behind the DP algorithm applied to discrete optimal control problems, see, for example, [25] or [26].

Solving the optimal control problem for the system (15) and the cost (8), i.e., finding the optimal h_k $k = 1, \ldots N_\tau$ and ν_0, requires the two variables t and ν to be represented by a discrete grid. The number of elements along each axis in the grid is defined by N_t and N_ν. If the optimal cost for a location in this grid for a specific k is denoted as $J_k^*(t_k, \nu_k)$, then the functional equation of DP in the forward direction can be written as

$$J_{k+1}^*(t_{k+1}, \nu_{k+1}) = \min_{h_k} [V_k(t_k, \nu_k, h_k) + J_k^*(t_k, \nu_k)]. \tag{16}$$

Here, V is the cost of moving from (t_k, ν_k) to (t_{k+1}, ν_{k+1}), i.e., the cost in (8), but with the integration from t_k to t_{k+1}. Then, for each (t_{k+1}, ν_{k+1}) point in the grid, compute J_{k+1}^*. Given the system described by (15), there is only one degree of freedom in J_{k+1}^* which is h_k. Even though the possible solutions may not coincide in specific (t_k, ν_k) grid points, it is still possible to interpolate values from J_k^*. Solving (16) for all k will result in the optimal cost for all the points in the grid. At the final iteration N_τ, the resulting matrix $J_{N_\tau}^*$ will hold the minimum cost for every grid point. Since t_k represents elapsed time, $t_{N_\tau} = t_f$, and as such the minimum cost for all execution times within the range of t_{N_τ} can be found based on the matrix $J_{N_\tau}^*$.

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on October 19,2026 at 19:03:37 UTC from IEEE Xplore. Restrictions apply.

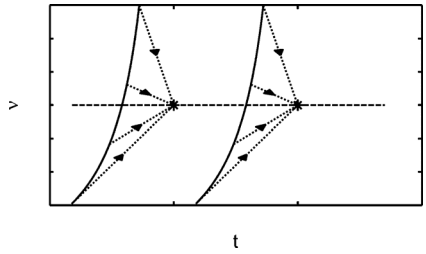


Fig. 2. The solid line denotes (t_k, ν_k) and the two points show (t_{k+1}, ν_{k+1}). Note that the solid line has the same form for all k. The dashed line represents a constant $\nu_{k+1} = 0.6$. The dotted lines illustrate that the point (t_{k+1}, ν_{k+1}) is reachable from (t_k, ν_k).

Before starting the optimization, the first and second derivative of q_0 with respect to τ are required. As previously mentioned, the velocity and acceleration of the time optimal trajectory can be used to define these. Since $\ddot{\tau}$ is defined as constant between the discrete time updates, this leads to every point (t_{k+1}, ν_{k+1}) in the grid having a corresponding (t_k, ν_k) curve from where that point can be accessed at step $k+1$. This movement in the grid is illustrated in Fig. 2. The minimization part in (16) consists of sampling along this curve and choosing the point where the combination of J_k^* and V_k yields the lowest cost. Since the sampling of J_k^* is not from specific grid points, be reminded that the cost will have to be interpolated.

D. Algorithm

The following will outline the algorithm. Note that t_k, h_k, ν_k, and $c1$ at lines 6, 7, and 9 in the routine OptimalSource are column vectors. The matrix multiplication will thus clone the vectors into matrices. A Matlab like notation is used where, for example, $A(:,1)$ corresponds to all elements in the first column of a matrix A and $A(1{:}2,1)$ the first two row elements in the first column.

Algorithm 1. Main dynamic programming loop

1: **procedure** DP-Algorithm ()
2: $J \leftarrow NaN$
3: $J(1{:}2,:,1) \leftarrow 0$ ▷ Set initial states to 0
4: **for** $k{:}=1$ **to** N_τ **do** ▷ N_τ iterations
5: $J(:,:,k+1) \leftarrow \mathrm{OptimalSource}(k, J(:,:,k))$
6: **end for**
7: **return** J
8: **end procedure**

In Algorithm 1, DP-Algorithm, the three-dimensional array J is used to store the optimal cost to reach a point in the grid for every time step. Its first two indices correspond to the values of the grid (transformed to integers) and the third the time step. The initial states are marked with zeros. For each step k along τ, call Algorithm 2, OptimalSource. This function will return the optimal value J_{k+1}^* for all points (t, ν) in the grid. This action is repeated N_τ number of times, i.e., the number of steps that should be taken along τ. When execution ends and J has been computed, the optimal cost for each time instance along the t axis of the grid can be found by retrieving the smallest element in $J(t,:,N_\tau)$. To find the optimal trajectory for a specific execution time, J can be used to trace the optimal path through the grid.

Algorithm 2. Optimal cost algorithm

1: **procedure** $\mathrm{OptimalSource}(k, J(:,:,k))$
2: **for** $e := 1$ **to** N_ν **do**
3: $[\nu_k, h_k] \leftarrow \mathrm{GridPoints}(\nu_{k+1}(e))$
4: $c1 \leftarrow \mathrm{CostFunction}(\nu_k, h_k, k)$
5: $t_{k+1} \leftarrow \mathrm{GetTimeIndices}(J(:,:,k))$
6: $T_k \leftarrow t_{k+1} * ones(1, N_\nu) - (h_k * ones(1, N_t))^T$
7: $X_2 \leftarrow (\nu_k * ones(1, N_t))^T$
8: $C2 \leftarrow \mathrm{Interpolate}(J(:,:,k), X_2, T_k)$
9: $C1 \leftarrow (c1 * ones(1, N_t))^T$
10: $C \leftarrow C1 + C2$
11: $f \leftarrow \mathrm{Min}(C, row)$
12: $J(:,e,k+1) \leftarrow \mathrm{FillUp}(f, t_k)$
13: **end for**
14: **return** $J(:,:,k+1)$
15: **end procedure**

The OptimalSource method works as follows. For each ν_{k+1} in the grid, N_ν number of times, the following is performed. Generate the possible relative grid points from where ν_{k+1} is reachable. Compute the cost to go (row 4) and which time indices in the grid that are relevant (row 5), i.e., discard time values that cannot be reached. Now, the current cost for all combinations of t_k and ν_k values is computed (rows 6–8) and added to the cost to go (rows 9–10). The summation on row 10 corresponds to that of V_k and J_k^* in (16), and choosing the minimum element relates to the minimization argument. These are the optimal costs for all t_{k+1} along the current ν_{k+1}. All that needs to be done now is to add NaN elements into f to correct for the time indices that were discarded on row 5. This result is then saved to $J(:,e,k+1)$.

E. Complexity

The shape of the (t_k, ν_k) curve, as shown in Fig. 2, does not change for different t_{k+1} along a constant ν_{k+1}, because (9) is time invariant. Since the scaling variable definition is time invariant, the cost function (8) is also time invariant, thus the cost function for a specific ν_{k+1} is a function of h_k and ν_k. This implies that the cost function does not have to be evaluated separately for each t_{k+1}, where the value of ν_{k+1} is the same. If instead, the original discrete system (12) had been used, each point in the τ/ν-grid would have to be evaluated separately.

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on October 19,2026 at 19:03:37 UTC from IEEE Xplore. Restrictions apply.

Since each iteration k is time invariant, the cost evaluations from previous iterations can be reused.

Using the reformulated method in (15), a total number of $(N_\tau - 1)$ iterations, of N_ν points connected to another N_ν points will be performed. As previously mentioned, the number of cost function evaluations does not increase with the size of N_t, while the number of interpolations from the grid increases by a factor N_ν. Thus, the number of cost function evaluations is relative to $(N_\tau - 1)N_\nu^2$, while the number of interpolations from the grid is relative to $N_t(N_\tau - 1)N_\nu^2$. Note that cost function evaluations are much more costly than interpolations.

For the original model (12), allowing reuse of cost function computations, the same expressions for cost function evaluations and interpolations are derived. But the size of N_τ and N_t are not the same as for the reformulated model (15). In the (12) case, t is a discrete variable and τ is a discrete approximation of a continuous variable. Thus, the resolution of τ must be higher than that of t. For (15), however, t is a discrete approximation, τ is discrete and the resolution of t needs to be finer than that of τ. From experience, a reasonable accuracy is achieved by an at least five times higher resolution for the discrete approximation. As such, (12) requires a minimum of five times as many cost function evaluations as (15).

Also, if one would like to minimize the computational requirements, using a nonuniform grid is a good idea. For long execution times, the value of h_k for each iteration will be larger, thus the resolution of the t axis can be lower for its upper range. On the other hand, long execution times imply small values and changes in ν, therefore a higher resolution is preferable at the lower range of ν. But if a variable resolution on t is used with 12, one cannot reuse cost function evaluations each iteration. This means that the additional performance gained from nonuniform gridding cannot be utilized by (12).

As previously mentioned, it would also be possible to use the model from [17] and solve it multiple times iterating the weighted cost function each time. Solving ONE trajectory planning instance of [17] has a time dependency relative to that of $(N_\tau - 1)N_\nu^2$. Note that we want to solve multiple instances, the number of instances is relative to that of the longest time scaling. The computational requirement for generating a complete energy function is therefore of order $N_i(N_\tau - 1)N_\nu^2$, where N_i is the number of executions needed to map the energy function.

For a comparison, it is reasonable to assume that the reformulated model (15) compared to the model in [17] will use the same size of N_τ, while N_ν is expected to be larger. The magnitude of the latter is dependent on the range of final times that are of interest. We assume a time scaling horizon of five times the original execution time and that N_ν does not need to be larger than twice the size of N_ν for [17]. This suggests that the number of cost function evaluations for the reformulated model (15) is about four times as many compared to [17]. Considering the additional interpolations adding approximately 30% computational time for our algorithm, a total of five times longer computation time is required for our method, solving one instance. However, five executions of [17] $(N_i = 5)$ are not enough to reach a sufficient resolution for the sought energy functions, thus making the reformulated model (15) in this paper more efficient for the generation of energy functions.

III. Energy Optimal Scheduling

With the energy function for each operation generated, an energy optimal schedule can be derived for the given process. The constraints governing the scheduling problem can be expressed with linear constraints consisting of real and binary variables. For a thorough account on mixed integer constraint modeling, see, for example, [27] or [28]. The decision variables for the problem are the real valued starting and stopping times for each robot operation as well as binary variables representing mutual exclusion. The energy function data from the DP algorithm are approximated as polynomials. These polynomials are checked for convexity and that the relative error related to the original data is within appropriate bounds. The cost function for the scheduling problem is now be formed using the polynomials. As such, the problem class is that of convex MINLP.

The MINLP master problem can be divided into a number of subproblems, one for each ordering combination. These are enumerated and each subproblem is treated as a linearly constrained problem with a nonlinear cost function. Not all of these choices are feasible, depending on the final time specified for the model. Feasible subproblems are solved using Matlab's optimization toolbox. The algorithm employed uses an interior point method combined with a barrier function for the constraints. The interior point method is described in [29], [30], and [31]. There are of course more efficient approaches to the master problem than explicit enumeration. Modern techniques for solving convex MINLP including among others: Branch-and-Bound, Outer-Approximation, LP/NLP-based branch and bound [32], [33].

A. Constraint Modeling

Let the global starting and finishing times for the jth operation executed by the ith robot be denoted t_{ij}^s and t_{ij}^f. A sequence of two operations is simply expressed by defining the finishing time of the preceding operation as smaller than the starting time of the following operation:

$$t_{ij}^s \geq t_{lm}^f + \epsilon \tag{17}$$

where l and m is the robot and operation indices of the first operation, i and j are that of the second, and ϵ a significantly small positive constant. It is also required to limit the minimum execution time of operations. If an operation j in robot i has a minimum execution time of $T_{0,ij}$, then the execution time can be constrained by

$$t_{ij}^f \geq t_{ij}^s + T_{0,ij}. \tag{18}$$

Shared resources can be expressed in the following way. If two operations, ij and lm share the same resource, then define $\sigma_{ij,lm}$ as a Boolean variable representing operation ij being performed after lm. Also, $\sigma_{lm,ij}$ is a Boolean variable representing the negation of the previous statement, i.e., operation lm is performed after ij. The resulting constraints for this example are

$$\begin{aligned} t_{ij}^s &\geq t_{lm}^f - M(1 - \sigma_{ij,lm}) \\ t_{lm}^s &\geq t_{ij}^f - M(1 - \sigma_{lm,ij}) \\ \sigma_{ij,lm} &+ \sigma_{lm,ij} = 1. \end{aligned} \tag{19}$$

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on October 19,2026 at 19:03:37 UTC from IEEE Xplore. Restrictions apply.

In other words, a Boolean variable or expression is used to negate constraints when false. As such, the constant M needs to be sufficiently large for this negation to be valid. The same principle can be used for one robot having an alternative order of execution for its operations.

For constraining the cycle time, an additional variable is added. This variable is constrained to be larger than the stopping time of the last operation for each robot. Adding an upper bound for the new variable will now constrain the complete cycle time of the cell. With the scheduling problem described by these mixed integer linear constraints and the cost function by the convex polynomials, the optimization model can be solved using any standard MINLP solver.

IV. Case Study

For the case study, an example with four industrial robots is considered. The four robots work together on a single work piece located in between the robots. There is a total number of 40 operations, all for which an energy function, $E(t_f)$, is computed. The cell includes three common zones in which only one robot can work at a time. Each robot has three tasks to perform including between one and five operations, most of which belong to common zones. Also, one of the tasks for each robot requires the other two tasks to be completed in order to be allowed to start. The scheduling problem has 82 real and 32 binary variables, and in total there are 1280 valid integer combinations. For a near time optimal cycle time, only 44 combinations were feasible. Note that if a robot is not performing any operation, there is still a cost based on the gravity term in (1), this is called the holding power. It can be added to the cost function just like the energy functions. For this time, however, only a first degree expression is needed as the holding power increases linearly with time.

The robot cell and robot operations were modeled using an offline programming and simulation environment for robot systems, ABB RobotStudio [23]. The software automatically generates time optimal trajectories for each robot operation. The time/position data of these operations can be extracted and then filtered to produce speed and acceleration. This filtered data forms the original trajectories that are scaled by our DP algorithm in Section II-D. The algorithm was implemented in Matlab with the output being energy function polynomials, as mentioned in Section III. The final MINLP scheduling problem was then solved, also using Matlab, as outlined in Section III.

All optimization was run on a Windows 7 64 bit system with a 2.66 [GHz] Intel Core2 Quad CPU and 4 [GB] of RAM. The final time feasibility check for each subproblem, which also generates a starting point, took <0.1 [s] in 98% of the cases. Most subproblems were solved in less than 10 [s]. It should be noted that for a few instances the initial barrier function weighting had to be varied in order for Matlab to produce an optimal solution. As for the minimum energy trajectory planning problem, with $N_\tau = 30$, $N_\nu = 50$, and $N_t = 186$, each operation instance was solved in close to 40 [s].

A. Results

A sample operation has been picked to illustrate the properties of dynamic scaling on individual operations. Fig. 3 illustrates the energy consumption profiles for three various execution times. The dotted curve belongs to the time optimal trajectory with an execution time of roughly 10 [s], the solid and dashed are the same trajectory but with the final time extended by factor a 1.75 and 2.75. Note how the first part of the curves has converged for the two scaled cases. This reduces the time optimal trajectory's characteristically initial high acceleration. The dashed curve has an almost constant power consumption between 5–15 [s], this is because during this segment of the path, the gravitational force affecting the manipulator is the lowest. Because of this the optimization results in a slow traversal of this segment, minimizing the cost of gravity. In Fig. 4, the resulting energy function for the sample operation is presented. The dashed line, which increases linearly, shows the time optimal energy cost and the holding power required to stay stationary for the remaining time. The dotted and solid curves are the energy cost for linear and dynamic scaling. Note that not all operations differ that much between linear and dynamic scaling, the average difference should be somewhat apparent from the overall result in Fig. 5. All the operations scaled resulted in convex functions as regenerative braking was not considered.

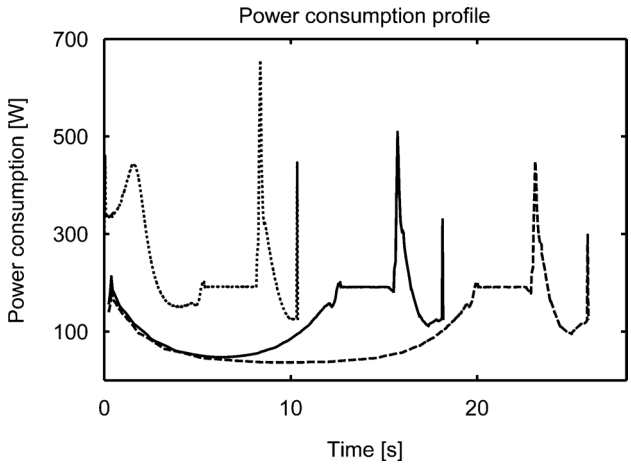


Fig. 3. The power consumption profiles for a sample operation of one six DOF industrial robot with three various execution times. The dotted curve belongs to the time optimal, the solid and dashed curves are the same trajectory but with the final time extended by factor 1.75 and 2.75.

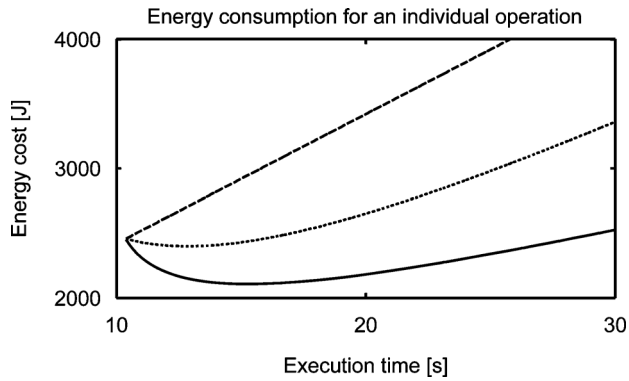


Fig. 4. Three energy functions based on the same sample operation, as in Fig. 3. The dashed line is based on time optimal execution and holding power required to stay stationary for the remaining time. The dotted and solid curves are the minimal energy costs for linear and dynamic scaling.

In Fig. 5, the total energy consumption for all four robots is shown, running an energy optimal schedule. The dashed line

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on October 19,2026 at 19:03:37 UTC from IEEE Xplore. Restrictions apply.

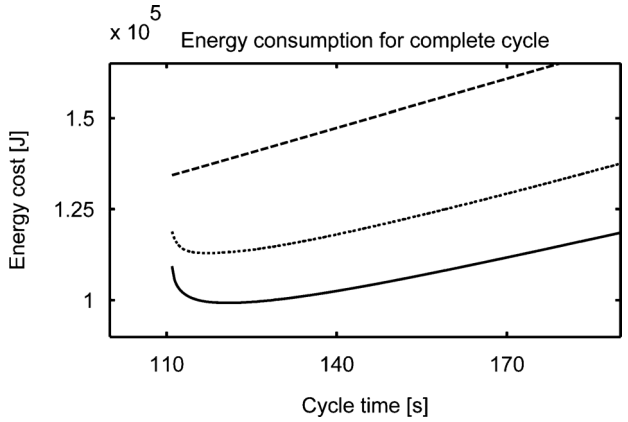


Fig. 5. Overall minimum energy consumption for the case study as a function of cycle time. The dashed line shows scheduling where no scaling is allowed, the dotted and solid curves represent scheduling based on linearly and dynamically scaled operations.

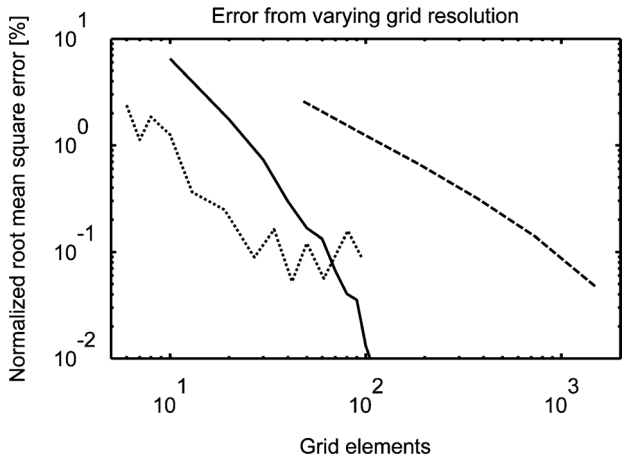


Fig. 6. The resulting normalized root mean square error for varied parameter values compared to a benchmark. The dotted curve is varied N_ν, with benchmark $N_\tau = 30$, $N_\nu = 119$, $N_t = 186$. The solid curve shows varied N_τ and $N_t = 6.2N_\tau$, benchmark $N_\tau = 120$, $N_\nu = 96$, $N_t = 744$. Last, the dashed curve is varied N_t, benchmark $N_\tau = 30$, $N_\nu = 50$, $N_t = 2961$.

shows the result of no scaling, i.e., minimum energy scheduling is performed but no scaling is allowed. The dotted and solid curves represent linear and dynamic scaling. While upholding a time optimal cycle time, linear scaling reduces the energy usage by 11%, and dynamic scaling a total of 18%. If the cycle time is allowed to be extended by 10% (10 [s]), linear scaling will reduce energy cost by 18%, and dynamic scaling as much as 28%.

For the sample operation considered in Fig. 3, an analysis of the root mean square error (RMSE) for varying parameter values can be found in Fig. 6. Top dotted curve shows variations in N_ν. For the solid, the resolution of τ is varied. Note that this also changes the resolution of t, $N_t = 6.2/N_\tau$. The dashed curve shows a varying N_t. All tests were performed for a five times longer execution time than the time optimal. For all three parameters, higher resolution decreases the RMSE. The error tends to zero for all but the dotted curve which stabilizes in the range of 0.1% RMSE. This can be contributed to numerical errors. We conclude that it is possible to run the algorithm at low resolutions with only relatively small errors. We estimate that the settings used for the case study, mentioned earlier, should not produce an error of more than 1% RMSE. In fact, N_ν could actually be lowered to about 20 without any significant impact.

V. Discussion and Conclusion

This paper presents a DP method, which can be used to find multiple energy optimal trajectories with varying execution times that follow the same path as a given trajectory. These minimum energy costs for a given execution times are then used to form the cost function used for scheduling of multiple robots working in the same shared space. The modification of the original trajectory is defined as a dynamic scaling. Previously, linear scaling of operations has been used for scheduling. A case study of four six-joint industrial robots with booking of common zones is presented in order to evaluate the possibility of energy reduction using this method. Results show that linear scaling can decrease the total energy cost by 10%–20%. Employing the dynamic scaling, as suggested in this paper, will further reduce the energy cost by 10%. Even though energy consumption for smaller robots can be considered marginal, larger industrial robots carrying heavy loads and other automated robotic machinery performing heavy lifting can benefit greatly from the suggested scheduled energy optimal trajectories.

References

[1] R. Isermann, *Mechatronic Systems: Fundamentals*. : Springer, 2005.
[2] R. Saidur, "A review on electrical motors energy use and energy savings," *Renewable and Sustainable Energy Reviews*, vol. 14, no. 3, pp. 877–898, 2010.
[3] R. Visinka, *Energy Efficient Three-Phase AC Motor Drives for Appliance and Industrial Applications,* ch. 2, Goldberg and Middleton, 2002.
[4] A. Yang, J. Pu, C. B. Wong, and P. Moore, "By-pass valve control to improve energy efficiency of pneumatic drive system," *Control Engineering Practice*, vol. 17, no. 6, pp. 623–628, 2009.
[5] G. Hirzinger, N. Sporer, A. Albu-Schaffer, M. Hahnle, R. Krenn, A. Pascucci, and M. Schedl, "Dlr's torque-controlled light weight robot iii-are we reaching the technological limits now?," in *Proc. IEEE Int. Conf. Robot. Autom., ICRA'02*, 2002, vol. 2, pp. 1710–1716.
[6] H. Diken, "Energy efficient sinusoidal path planning of robot manipulators," *Mech. Mach. Theory*, vol. 29, no. 6, pp. 785–792, 1994.
[7] J. S. Park, "Motion profile planning of repetitive point-to-point control for maximum energy conversion efficiency under acceleration conditions," *Mechatronics*, vol. 6, no. 6, pp. 649–663, 1996.
[8] E. S. Sergaki, G. S. Stavrakakis, and A. D. Pouliezos, "Optimal robot speed trajectory by minimization of the actuator motor electromechanical losses," *J. Intell. Robot. Syst.*, vol. 33, pp. 187–207, Feb. 2002.
[9] O. Maimon, E. Profeta, and S. Singer, "Energy analysis of robot task motions," *J. Intell. Robot. Syst.*, vol. 4, pp. 175–198, 1991.
[10] F. Roos, H. Johansson, and J. Wikander, "Optimal selection of motor and gearhead in mechatronic applications," *Mechatronics*, vol. 16, no. 1, pp. 63–72, 2006.
[11] T. Izumi, H. Zhou, and Z. Li, "Optimal design of gear ratios and offset for energy conservation of an articulated manipulator," *IEEE Trans. Autom. Sci. Eng.*, vol. 6, no. 3, pp. 551–557, Jul. 2009.
[12] S. Panek, O. Stursberg, and S. Engell, "Optimization of timed automata models using mixed-integer programming," in *In Formal Modeling And Analysis of Timed Systems*. New York: Springer, 2004, vol. 2791, LNCS, pp. 73–87.
[13] A. Kobetski and M. Fabian, "Time-optimal coordination of flexible manufacturing systems using deterministic finite automata and mixed integer linear programming," *Discrete Event Dynamic Syst.*, vol. 19, pp. 287–315, Sep. 2009.
[14] A. Kobetski and M. Fabian, "Velocity balancing in flexible manufacturing systems," in *Proc. 9th Int. Workshop Discrete Event Syst., WODES'08*, May 2008, pp. 358–363.
[15] A. Vergnano, C. Thorstensson, B. Lennartson, P. Falkman, M. Pellicciari, C. Yuan, S. Biller, and F. Leali, "Embedding detailed robot energy optimization into high-level scheduling," in *Proc. IEEE Conf. Autom. Sci. Eng., CASE'10*, 2010, pp. 386–392.

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on October 19,2026 at 19:03:37 UTC from IEEE Xplore. Restrictions apply.

[16] O. Wigström and B. Lennartson, "Energy optimization of trajectories for high level scheduling," in *Proc. IEEE Conf. Autom. Sci. Eng., CASE'11*, 2011, pp. 654–659.
[17] K. Shin and N. McKay, "A dynamic programming approach to trajectory planning of robotic manipulators," *IEEE Trans. Autom. Control*, vol. 31, no. 6, pp. 491–500, Jun. 1986.
[18] G. Field and Y. Stepanenko, "Iterative dynamic programming: An approach to minimum energy trajectory planning for robotic manipulators," in *Proc. IEEE Int. Conf. Robot. Autom.*, Apr. 1996, vol. 3, pp. 2755–2760.
[19] B. J. Martin and J. E. Bobrow, "Minimum effort motions for open chain manipulators with task-dependent end-effector constraints," in *Proc. Int. Conf. Robot. Autom.*, Apr. 1997, vol. 3, pp. 2044–2049.
[20] Z. Shiller, "Time-energy optimal control of articulated systems with geometric path constraints," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 1994, vol. 4, pp. 2680–2685.
[21] A. Gasparetto and V. Zanotto, "A technique for time-jerk optimal planning of robot trajectories," *Robot. Comput.-Integr. Manuf.*, vol. 24, no. 3, pp. 415–426, 2008.
[22] L. Sciavicco, B. Siciliano, and B. Sciavicco, *Modelling and Control of Robot Manipulators*, 2nd ed. Secaucus, NJ: Springer-Verlag, 2000.
[23] "ABB RobotStudio," 2011. [Online]. Available: http://www.robotstudio.com
[24] D. Y. Ohm, *Selection of Servo Motors and Drives (Rev.2),* 2006. [Online]. Available: http://www.drivetechinc.com
[25] A. E. Bryson and Y. C. Ho, *Applied Optimal Control; Optimization, Estimation, and Control*, A. E. Bryson, Jr. and Y. C. Ho, Eds. Waltham, MA: Blaisdell, 1969.
[26] D. S. Naidu, *Optimal Control Systems*, ser. Electrical engineering textbook. Boca Raton, FL: CRC Press, 2003.
[27] Y. Pochet and L. A. Wolsey, *Production Planning by Mixed Integer Programming*, ser. Springer Series in Operations Research and Financial Engineering. Secaucus, NJ: Springer-Verlag , 2006.
[28] H. P. Williams, *Model Building in Mathematical Programming*, 4th ed. New York: Wiley, 1999.
[29] R. H. Byrd, J. C. Gilbert, and J. Nocedal, "A trust region method based on interior point techniques for nonlinear programming," in *Math. Program.*, 2000, vol. 89, pp. 149–185, 10.1007/PL00011391.
[30] R. H. Byrd, M. E. Hribar, and J. Nocedal, "An interior point algorithm for large scale nonlinear programming," *SIAM J. Opt.*, vol. 9, pp. 877–900, 1997.
[31] R. A. Waltz, J. L. Morales, J. Nocedal, and D. Orban, "An interior algorithm for nonlinear optimization that combines line search and trust region steps," *Math. Program.*, vol. 107, pp. 391–408, Jul. 2006.
[32] F. P. Fernandes, M. F. P. Costa, and E. M. G. P. Fernandes, "Overview on mixed integer nonlinear programming problems," in *Proc. AIP Conf.*, 2009, vol. 1168, pp. 1374–1377.
[33] S. Leyffer, J. Linderoth, J. Luedtke, A. Miller, and T. Munson, "Applications and algorithms for mixed integer nonlinear programming," in *J. Phys.: Conference Series*, 2009, vol. 180, pp. 012014–012014.



Oskar Wigström was born in Gothenburg, Sweden, in 1986. He received the M.Sc. degree in systems, control and mechatronics, majoring in automatic control from the Chalmers University of Technology, Gothenburg, Sweden, in 2010. He is currently working towards the Ph.D. degree in automation at the Chalmers University of Technology.

His current research interests include nonlinear optimization, hybrid systems, robust scheduling, and integrated optimization methods.



Bengt Lennartson (M'10) was born in Gnosjö, Sweden, in 1956. He received the Ph.D. degree in automatic control from the Chalmers University of Technology, Gothenburg, Sweden, in 1986.

Since 1999, he has been a Professor of the Chair of Automation, Department of Signals and Systems, Chalmers University of Technology, and was Dean of Education from 2004 to 2007. Since 2005, he has been a Guest Professor with University West, Trollhättan, Sweden. He is (co)author of two books and 180 peer reviewed international papers with more than 2200 citations. His main areas of interest include discrete event and hybrid systems, especially for manufacturing applications, as well as robust feedback control.

Dr. Lennartson was the Chairman of the Ninth International Workshop on Discrete Event Systems, WODES'08, and currently is a member of the Advisory Board for IEEE TRANSACTIONS ON AUTOMATION, SCIENCE AND ENGINEERING (TASE). He was an Associate Editor for *Automatica*.



Alberto Vergnano received the M.S. degree in mechanical engineering and the the Ph.D. degree in high mechanics and automotive design and technology from the University of Modena and Reggio Emilia, Modena, Italy, in 2006 and 2010, respectively.

Currently, he is an Assistant Professor with the Department of Mechanical and Civil Engineering, University of Modena and Reggio Emilia. His research interests are methods for virtual commissioning of automatic machines, integrated product and process development, and energy consumption reduction in industrial robotic systems.



Claes Breitholtz received the Ph.D. degree from the Chalmers University of Technology, Göteborg, Sweden, in 1982.

He is a Professor of Automatic Control in the Department of Signals and Systems at the Chalmers University of Technology in Göteborg. After receiving the Ph.D. degree he left academia for a position at ASEA (now ABB) in Västerås, where he was engaged in industrial applications of adaptive control (in the NOVATUNE project) until 1985. After one year as a consultant engineer, he returned to the Chalmers University of Technology in 1986 as an Associate Professor of Automatic Control and became a Professor in 2001. His research interests are in the areas of optimal control, nonlinear control and model-based control of distributed parameter systems. Among control applications, iron ore pelletizing plants, packed bed distillation, biofilms in wastewater treatment plants, chemical reactors, and fighter jet engines, can be mentioned.

Authorized licensed use limited to: Technische Universitaet Muenchen. Downloaded on October 19,2026 at 19:03:37 UTC from IEEE Xplore. Restrictions apply.